

Experiment – 8

1. **Aim:** To implement Supervised Learning algorithm - Random Forest.
2. **Objectives:** Students should be familiarize with Learning Architectures and Frameworks
3. **Prerequisite:** Python basics

4. Pre-Experiment Exercise:

Theory:

Random Forest Algorithm

Decision trees involve the greedy selection of the best split point from the dataset at each step.

This algorithm makes decision trees susceptible to high variance if they are not pruned. This high variance can be harnessed and reduced by creating multiple trees with different samples of the training dataset (different views of the problem) and combining their predictions. This approach is called bootstrap aggregation or bagging for short.

A limitation of bagging is that the same greedy algorithm is used to create each tree, meaning that it is likely that the same or very similar split points will be chosen in each tree making the different trees very similar (trees will be correlated). This, in turn, makes their predictions similar, mitigating the variance originally sought.

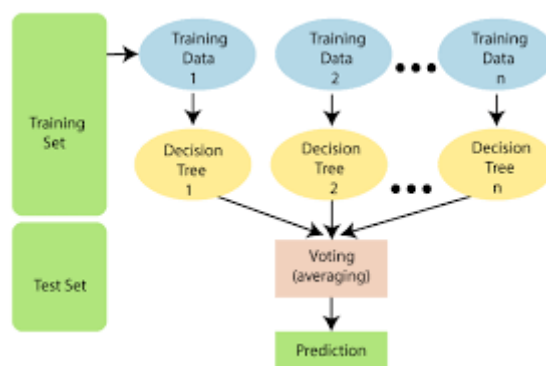
We can force the decision trees to be different by limiting the features (rows) that the greedy algorithm can evaluate at each split point when creating the tree. This is called the Random Forest algorithm.

Like bagging, multiple samples of the training dataset are taken and a different tree trained on each. The difference is that at each point a split is made in the data and added to the tree, only a fixed subset of attributes can be considered.

For classification problems, the type of problems we will look at in this tutorial, the number of attributes to be considered for the split is limited to the square root of the number of input features.

$\text{num_features_for_split} = \sqrt{\text{total_input_features}}$

The result of this one small change are trees that are more different from each other (uncorrelated) resulting predictions that are more diverse and a combined prediction that often has better performance than single tree or bagging alone.



6. Laboratory Exercise

Procedure

- Use google colab for programming.
- Import required packages.
- Demonstrate random forest classifier for any given dataset.
- Add relevant comments in your programs and execute the code. Test it for various cases.

Post-Experiments Exercise:

A. Extended Theory:

- Write real life applications of Random Forest Classifier.

B. Conclusion:

- Write what was performed in the program (s) .
- What is the significance of program and what Objective is achieved?

C. References:

- <https://machinelearningmastery.com/implement-random-forest-scratch-python/>.
- <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/>

Laboratory Exercise

Procedure

RANDOM FOREST

```
# Dataset: Breast Cancer Wisconsin Dataset
# Step 1: Import required packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Step 2: Load the dataset
cancer = load_breast_cancer()

# Create a DataFrame for better understanding
data = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data['target'] = cancer.target
data['target_name'] = data['target'].apply(lambda x: cancer.target_names[x])

print("Dataset Shape:", data.shape)
print("Classes:", cancer.target_names)
print("\nSample Data:\n", data.head())
```

Dataset Shape: (569, 32)
Classes: ['malignant' 'benign']

Sample Data:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension ...	worst perimeter	worst area	worst smoothness \
0	0.07871 ...	184.60	2019.0	0.1622
1	0.05667 ...	158.80	1956.0	0.1238
2	0.05999 ...	152.50	1709.0	0.1444
3	0.09744 ...	98.87	567.7	0.2098
4	0.05883 ...	152.20	1575.0	0.1374

	worst compactness	worst concavity	worst concave points	worst symmetry \
0	0.6656	0.7119	0.2654	0.4601
1	0.1866	0.2416	0.1860	0.2750
2	0.4245	0.4504	0.2430	0.3613
3	0.8663	0.6869	0.2575	0.6638
4	0.2050	0.4000	0.1625	0.2364

	worst fractal dimension	target	target_name
0	0.11890	0	malignant
1	0.08902	0	malignant
2	0.08758	0	malignant
3	0.17300	0	malignant
4	0.07678	0	malignant

[5 rows x 32 columns]

How can I

```
# Step 3: Split the data into training and testing sets
X = cancer.data
y = cancer.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Create and train the Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = rf_model.predict(X_test)
```

RandomForestClassifier ⓘ ⓘ
RandomForestClassifier(random_state=42)

```
# Step 6: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(
    y_test,
    y_pred,
    target_names=cancer.target_names,
    output_dict=True
)

print("\n✅ Model Evaluation Results:")
print("Accuracy Score:", accuracy)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=cancer.target_names))
```

STEP 6 Output:

✅ Model Evaluation Results:
Accuracy Score: 0.9649122807017544

Confusion Matrix:
[[40 3]
 [1 70]]

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.93	0.95	43
benign	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

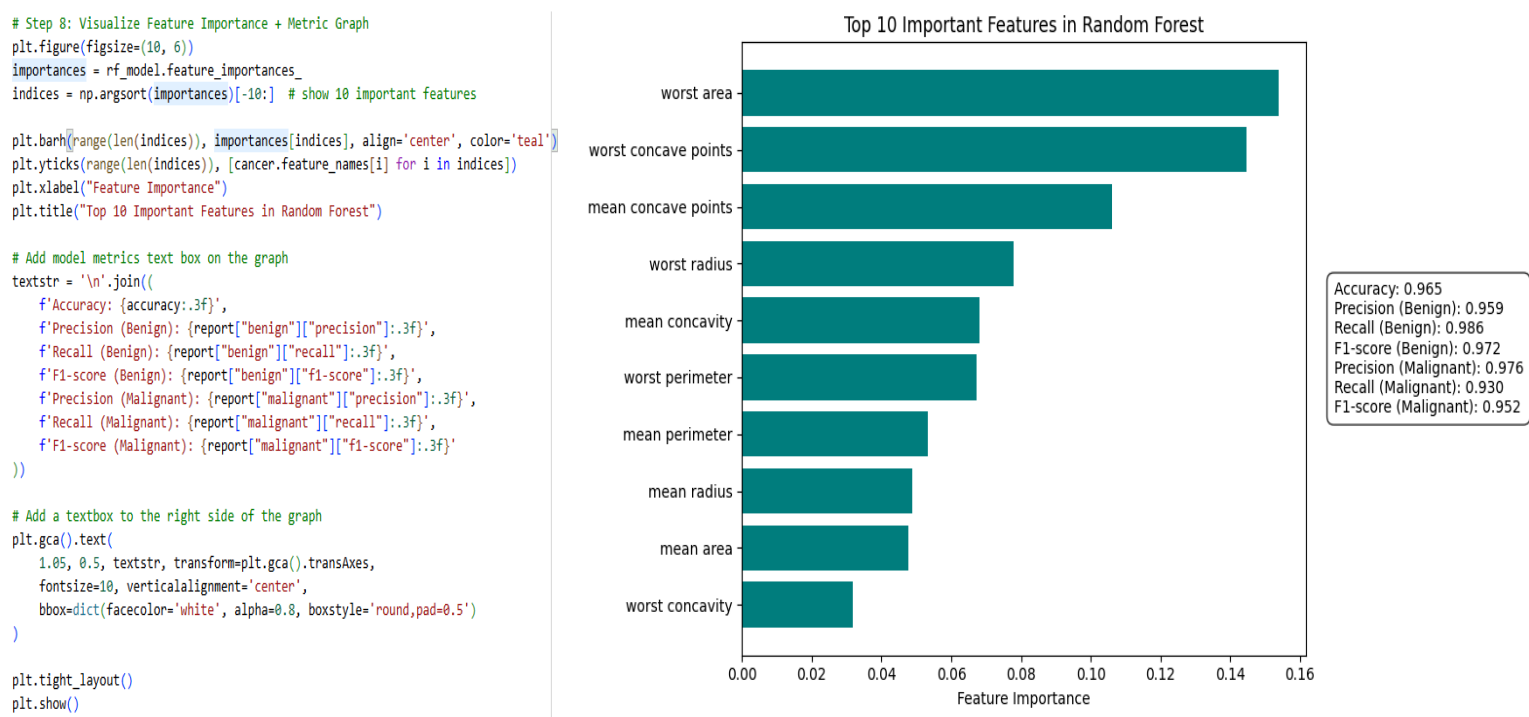
STEP 7:

```
# Step 7: Test with various cases
sample_cases = X_test[:3]
predicted = rf_model.predict(sample_cases)

print("\n🔍 Test Case Predictions:")
for i, pred in enumerate(predicted):
    print(f"Case {i+1} --> Predicted: {cancer.target_names[pred]}")

🔍 Test Case Predictions:
Case 1 --> Predicted: benign
Case 2 --> Predicted: malignant
Case 3 --> Predicted: malignant
```

VISUALIZATION CODE AND OUTPUT:



DECISION TREE

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the dataset
cancer = load_breast_cancer()

# Create a DataFrame for better visualization
data = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data['target'] = cancer.target
data['target_name'] = data['target'].apply(lambda x: cancer.target_names[x])

print("\n Dataset Shape:", data.shape)
print("\n Classes:", cancer.target_names)
print("\nSample Data:\n", data.head())
```

Dataset Shape: (569, 32)

Classes: ['malignant' 'benign']

Sample Data:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	
2	0.15990	0.1974	0.12790	0.2069	
3	0.28390	0.2414	0.10520	0.2597	
4	0.13280	0.1980	0.10430	0.1809	

	mean fractal dimension	...	worst perimeter	worst area	worst smoothness	\
0	0.07871	...	184.60	2019.0	0.1622	
1	0.05667	...	158.80	1956.0	0.1238	
2	0.05999	...	152.50	1709.0	0.1444	
3	0.09744	...	98.87	567.7	0.2098	
4	0.05883	...	152.20	1575.0	0.1374	

	worst compactness	worst concavity	worst concave points	worst symmetry	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	worst fractal dimension	target	target_name
0	0.11890	0	malignant
1	0.08902	0	malignant
2	0.08758	0	malignant
3	0.17300	0	malignant
4	0.07678	0	malignant

[5 rows x 32 columns]

```
# Split the data into training and testing sets
X = cancer.data
y = cancer.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the Decision Tree Classifier
dt_model = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_model.fit(X_train, y_train)

# Make predictions
y_pred = dt_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=cancer.target_names, output_dict=True)

print("\n Model Evaluation Results:")
print("Accuracy Score:", accuracy)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=cancer.target_names))
```

Model Evaluation Results:

Accuracy Score: 0.9473684210526315

Confusion Matrix:

[[38 5]
[1 70]]

Classification Report:

	precision	recall	f1-score	support
malignant	0.97	0.88	0.93	43
benign	0.93	0.99	0.96	71
accuracy			0.95	114
macro avg	0.95	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114

```
# Test with sample cases
sample_cases = X_test[:3]
predicted = dt_model.predict(sample_cases)

print("\n🔍 Test Case Predictions:")
for i, pred in enumerate(predicted):
    print(f"Case {i+1} --> Predicted: {cancer.target_names[pred]}")

# Visualize Feature Importance + Metric Graph
plt.figure(figsize=(10, 6))
importances = dt_model.feature_importances_
indices = np.argsort(importances)[-10:] # Top 10 important features

plt.barh(range(len(indices)), importances[indices], align='center', color='orange')
plt.yticks(range(len(indices)), [cancer.feature_names[i] for i in indices])
plt.xlabel("Feature Importance")
plt.title("Top 10 Important Features in Decision Tree")

# Add metrics textbox
textstr = '\n'.join((
    f'Accuracy: {accuracy:.3f}',
    f'Precision (Benign): {report["benign"]["precision"]:.3f}',
    f'Recall (Benign): {report["benign"]["recall"]:.3f}',
    f'F1-score (Benign): {report["benign"]["f1-score"]:.3f}',
    f'Precision (Malignant): {report["malignant"]["precision"]:.3f}',
    f'Recall (Malignant): {report["malignant"]["recall"]:.3f}',
    f'F1-score (Malignant): {report["malignant"]["f1-score"]:.3f}'
))

plt.gca().text(
    1.05, 0.5, textstr, transform=plt.gca().transAxes,
    fontsize=10, verticalalignment='center',
    bbox=dict(facecolor='white', alpha=0.8, boxstyle='round,pad=0.5')
)

plt.tight_layout()
plt.show()
```

How can I ins

🔍 Test Case Predictions:
Case 1 --> Predicted: benign
Case 2 --> Predicted: malignant
Case 3 --> Predicted: malignant

